

Name: Rahul Bhati, 2223416 , LY-AIEC

Subject: High Performance Computing (21BTCS030)

Theory Assignment-2

Q. 1 Calculate GPU Tensor Cores FLOPs for Nvidia V100 and A100 Cards.

1. NVIDIA Tesla V100 (Volta architecture)

- **GPU Specs:**
 - 640 Tensor Cores
 - Base clock ≈ 1.45 GHz
 - FP16 Tensor performance: **~ 125 TFLOPS**
- **Calculation:**

Each Tensor Core performs a 4×4 matrix multiply and accumulate (MAC) per cycle:

64 FMA per clock per Tensor Core = 128 FLOPs per clock per Tensor Core

Total FLOPs = 640 Tensor Cores $\times 128 \times 1.45 \times 10^9 \approx 1.19 \times 10^{14}$ FLOPs = $119 TFLOPS$

 **Reported Peak: ~ 125 TFLOPS FP16 Tensor performance**

(Matches NVIDIA's official datasheet)

2. NVIDIA A100 (Ampere architecture)

- **GPU Specs (40GB or 80GB models):**
 - 432 Tensor Cores
 - Boost clock ≈ 1.41 GHz
 - Supports TF32, FP16, BF16, FP64 Tensor operations
 - FP16 Tensor performance: **~ 312 TFLOPS**
- **Calculation (FP16 example):**

Total FLOPs = 432 Tensor Cores $\times 256 \times 1.41 \times 10^9 \approx 1.56 \times 10^{14}$ FLOPs = $312 TFLOPS$

 **Reported Peak:**

- **312 TFLOPS FP16 Tensor**
- **156 TFLOPS BF16**
- **19.5 TFLOPS FP64 (non-Tensor)**
- **156 TFLOPS TF32 Tensor**

Q. 2 a. Find GDDR2, GDDR3, GDDR4, GDDR5 and GDDR6 Memory Interface Width, Frequency, and Also find Ops on clock rise and fall.

1 GDDR2 (Graphics DDR2)

Parameter	Description / Value
Interface Width (per chip)	32-bit
Typical Frequency (Data Rate)	500–1000 MHz (1.0–2.0 Gbps effective)
Operations per Clock	2 ops/clock (1 on rising + 1 on falling edge)
Type	Double Data Rate (DDR)

Explanation:

GDDR2 transfers **data on both rising and falling edges** of the clock → **2 operations per clock**.

2 GDDR3

Parameter	Description / Value
Interface Width (per chip)	32-bit
Typical Frequency (Data Rate)	1.0–1.5 GHz (2.0–3.0 Gbps effective)
Operations per Clock	2 ops/clock (DDR)
Type	Double Data Rate (DDR)

 **GDDR3** improved signal integrity and lower power but same DDR principle — **2 transfers per clock**.

3 GDDR4

Parameter	Description / Value
Interface Width (per chip)	32-bit
Typical Frequency (Data Rate)	1.6–2.0 GHz (3.2–4.0 Gbps effective)
Operations per Clock	2 ops/clock (DDR)

 Introduced higher prefetch (8n) and better interface signaling — still **2 operations per clock**.

4 GDDR5

Parameter	Description / Value
Interface Width (per chip)	32-bit
Typical Frequency (Data Rate)	3.5–4.0 GHz (7.0–8.0 Gbps effective)
Operations per Clock	4 ops/clock (QDR equivalent)
Type	Quad Data Rate signaling

→ **GDDR5** doubles data transfers per clock edge compared to DDR — effectively **4 operations per clock** (2 per edge × differential signaling per lane).

5 GDDR6

Parameter	Description / Value
Interface Width (per chip)	32-bit, internally split into two 16-bit channels
Typical Frequency (Data Rate)	8–14 GHz (16–28 Gbps effective)
Operations per Clock	4 ops/clock per channel
Type	Quad Data Rate (per channel)

→ Each **GDDR6 chip** has **two independent 16-bit channels**, each running at **double data rate** on both edges — giving **4 total operations per clock per channel**.

b. Calculate the global block id of following blocks B(1,0,0) and B(3,0,0).

✖ Given:

We are asked to find the **global block ID** for:

- B(1, 0, 0)
- B(3, 0, 0)

But — to compute this, we **must know** the **grid dimensions**, because CUDA computes the **global block index** based on how many blocks exist in each dimension.

So let's assume the **standard CUDA indexing rule**:

$$\text{Global Block ID} = \text{blockIdx}.x + \text{blockIdx}.y \times \text{gridDim}.x + \text{blockIdx}.z \times \text{gridDim}.x \times \text{gridDim}.y$$

⚙ Assume a 1D Grid

If the grid is **1-dimensional** (common case), then:

$$\text{Global Block ID} = \text{blockIdx}.x$$

So:

- For B(1, 0, 0) → Global Block ID = 1
- For B(3, 0, 0) → Global Block ID = 3

✅ Answer (for 1D grid):

Block	Global Block ID
-------	-----------------

B(1,0,0)	1
----------	---

B(3,0,0)	3
----------	---

⚙ If it's a 2D Grid

Then you need **gridDim.x** (the number of blocks in X direction).

$$\text{Global Block ID} = \text{blockIdx}.x + \text{blockIdx}.y \times \text{gridDim}.x$$

If for example **gridDim.x = 4**, then:

Block	Calculation	Global Block ID
-------	-------------	-----------------

B(1,0,0)	1 + 0×4	1
----------	---------	---

Block Calculation Global Block ID

B(3,0,0) $3 + 0 \times 4 = 3$

Same as before since $\text{blockIdx.y} = 0$.

⚙️ If it's a 3D Grid

Then:

$$\text{Global Block ID} = \text{blockIdx.x} + \text{blockIdx.y} \times \text{gridDim.x} + \text{blockIdx.z} \times \text{gridDim.x} \times \text{gridDim.y}$$

Since both $y=0$ and $z=0$, this again simplifies to blockIdx.x .

✅ Final Answer:

$\text{Global Block ID}(1,0,0) = 1, \text{Global Block ID}(3,0,0) = 3$
--

Q. 3 a. Calculate the global block id of following blocks B(1,1,0), B(2,1,0), and B(1,2,0).

⚙️ Formula for Global Block ID in CUDA:

$$\text{Global Block ID} = \text{blockIdx.x} + \text{blockIdx.y} \times \text{gridDim.x} + \text{blockIdx.z} \times \text{gridDim.x} \times \text{gridDim.y}$$

where:

- $\text{blockIdx} \rightarrow$ index of the block in each dimension (x, y, z)
 - $\text{gridDim} \rightarrow$ number of blocks in each grid dimension
-

⚙️ Given Blocks:

- B(1,1,0)
- B(2,1,0)
- B(1,2,0)

We need gridDim.x and gridDim.y to compute the IDs.

Let's assume a 2D grid with:

$$\text{gridDim.x} = 4, \text{gridDim.y} = 3$$

(this is a typical assumption unless specified otherwise)

 **Now Calculate:**

1 B(1,1,0)

$$ID = 1 + (1 \times 4) + (0 \times 4 \times 3) = 1 + 4 = \boxed{5}$$

2 B(2,1,0)

$$ID = 2 + (1 \times 4) + (0 \times 4 \times 3) = 2 + 4 = \boxed{6}$$

3 B(1,2,0)

$$ID = 1 + (2 \times 4) + (0 \times 4 \times 3) = 1 + 8 = \boxed{9}$$

 **Final Answers (for gridDim.x = 4):**

Block Global Block ID

B(1,1,0) 5

B(2,1,0) 6

B(1,2,0) 9

b. For Matrix Multiplication using multiple blocks, should we use 8X8, 16X16 or 32X32 blocks for GPU A100 (2048 Threads per SM and 32 Blocks per SM)? Find best block size.

 **1** Given A100 GPU Specs (key scheduling limits)

Parameter	Value
Max Threads per SM	2048 threads
Max Blocks per SM	32 blocks
Warp size	32 threads
Threads per Block (limit) ≤ 1024 threads	
SM Count	(varies by model — not needed here)

 **2** Candidate Block Sizes

Block Size Threads per Block

8×8	64 threads
16×16	256 threads
32×32	1024 threads

 **3** Compute How Many Such Blocks Can Fit per SM

We must satisfy both constraints:

- ≤ 2048 total threads per SM
- ≤ 32 blocks per SM

Let's calculate for each case:

Block Size	Threads per Block	Max Blocks (by threads) = $2048 / \text{ThreadsPerBlock}$	Limited by Blocks-per-SM (≤ 32)?	Active Blocks/SM	Active Threads/SM
8×8 (64)	64	$2048 / 64 = 32$	✓ ≤ 32	32	$32 \times 64 = 2048$
16×16 (256)	256	$2048 / 256 = 8$	✓ ≤ 32	8	$8 \times 256 = 2048$
32×32 (1024)	1024	$2048 / 1024 = 2$	✓ ≤ 32	2	$2 \times 1024 = 2048$

✓ All configurations fully use 2048 threads per SM.
However, occupancy and efficiency differ.

4 Compare Warp Utilization and Scheduling Efficiency

Block Size	Threads/Block	Warps/Block (Threads/32)	Active Warps/SM	Observations
8×8	64	2	$32 \times 2 = 64$ warps	Many small blocks → more scheduling overhead. Good latency hiding.
16×16	256	8	$8 \times 8 = 64$ warps	Balanced. Excellent occupancy and compute/memory balance.
32×32	1024	32	$2 \times 32 = 64$ warps	Full thread utilization, but fewer blocks → less flexibility for scheduling and shared memory use.

5 Shared Memory & Cache Considerations

- Each block in matrix multiplication typically loads a tile of A and B into shared memory.
- Shared memory per SM (A100): ~164 KB.
- For large blocks (e.g., 32×32), shared memory demand can be high:
 - Two 32×32 tiles of floats:
$$2 \times 32 \times 32 \times 4 = 8,192 \text{ bytes} = 8KB$$
 - That's fine, but per SM multiple such blocks may reduce concurrency.

Thus:

- 8×8 → too small; higher kernel overhead, low arithmetic intensity.
- 16×16 → ideal balance between occupancy, memory efficiency, and compute intensity.
- 32×32 → very large; fewer concurrent blocks, less scheduling flexibility (especially for smaller matrices).

Q. 4 a. Describe the improved method for computing a global sum using multiple cores and explain why it is more efficient than having a single master core perform all additions.

🧩 Problem Overview

We want to compute the sum of a large array (say N elements) using many cores (GPU threads or CPU cores).

Naively, one could let a single master core do all the additions:

$$\text{Sum} = a_1 + a_2 + a_3 + \dots + a_N$$

...but that means the master core performs $O(N)$ operations while other cores stay idle → very inefficient.

⚙️ 1 Naive Method (Single Master Core)

- Every thread reads its data but only one core does all the additions.
- Time complexity: $O(N)$ (serial addition).
- Other cores idle → wasted parallel hardware.
- Poor memory bandwidth utilization.

⚙️ 2 Improved Method — *Parallel Reduction (Tree-Based Summation)*

Instead of one master doing all work, we let all cores participate in a hierarchical (tree-like) reduction.

◆ Step-by-Step Process:

1. Divide the data among all threads.
 - Each thread (or block) computes a partial sum of its assigned elements in parallel.
2. Perform local reduction within each block.
 - Threads within a block use shared memory to cooperatively sum their chunk.
 - This is done in $\log_2(n)$ steps:
 - For stride = $n/2$ down to 1:
 - if `threadIdx < stride`:
 - `sdata[threadIdx] += sdata[threadIdx + stride]`
3. Write the block's partial sum to global memory.
 - Each block outputs one value.
4. Launch a second kernel (or recursive step) to sum all partial results (again using the same parallel reduction method).

 Example:

For 8 elements:

Step 1: a0 a1 a2 a3 a4 a5 a6 a7

Step 2: (a0+a1) (a2+a3) (a4+a5) (a6+a7)

Step 3: ((a0+a1)+(a2+a3)) ((a4+a5)+(a6+a7))

Step 4: Total = ((a0+a1+a2+a3)+(a4+a5+a6+a7))

Each level halves the number of active threads $\rightarrow \log_2(N)$ steps total.

b. Write a program which explains the usage of MPI sendrecv instructions for a simple exchange of data between two processes.

Goal: Demonstrate how two processes can exchange data simultaneously using the MPI_Sendrecv() instruction.

 Program: Simple Data Exchange Using MPI_Sendrecv

// Q4(b) - MPI Sendrecv Example

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int rank, size;
```

```
    int send_data, recv_data;
```

```
    MPI_Init(&argc, &argv);          // Initialize MPI
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get process rank
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total number of processes
```

```
    if (size < 2) {
```

```
        if (rank == 0)
```

```
            printf("Run with at least 2 processes.\n");
```

```
        MPI_Finalize();
```

```

    return 0;
}

// Each process sends its rank and receives from the other
send_data = rank;

if (rank == 0) {
    MPI_Sendrecv(&send_data, 1, MPI_INT, 1, 0,
                 &recv_data, 1, MPI_INT, 1, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 0 sent %d and received %d\n", send_data, recv_data);
} else if (rank == 1) {
    MPI_Sendrecv(&send_data, 1, MPI_INT, 0, 0,
                 &recv_data, 1, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process 1 sent %d and received %d\n", send_data, recv_data);
}

MPI_Finalize();
return 0;
}

```

Explanation

- `MPI_Sendrecv()` allows simultaneous send and receive, avoiding deadlocks.
- Syntax:
- `MPI_Sendrecv(sendbuf, sendcount, sendtype, dest, sendtag,`
- `recvbuf, recvcount, recvtype, source, recvtag,`
- `comm, status)`
- Both processes send and receive at the same time, so no one waits indefinitely.

Sample Output (using 2 processes)

```
$ mpirun -np 2 ./a.out
```

Process 0 sent 0 and received 1

Process 1 sent 1 and received 0

c. MPI program to send and receive Hello World messages from all other processes to a Root process and print the received messages.

Goal:

Each process sends a “Hello World” message to the root process (rank 0), and the root process receives and prints all messages.

✓ Program: Send/Receive “Hello World” Messages

// Q4(c) - MPI Hello World Send/Receive Example

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int rank, size;
```

```
    char message[50];
```

```
    MPI_Status status;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    if (rank == 0) {
```

```
        printf("Root process waiting for messages...\n");
```

```
    // Root receives from all other processes
```

```
    for (int i = 1; i < size; i++) {
```

```
        MPI_Recv(message, 50, MPI_CHAR, i, 0, MPI_COMM_WORLD, &status);
```

```
        printf("Received from process %d: %s\n", i, message);
```

```
    }  
} else {  
    // Each non-root process sends a message to root  
    sprintf(message, "Hello World from process %d", rank);  
    MPI_Send(message, strlen(message) + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);  
}  
  
MPI_Finalize();  
return 0;  
}
```

Explanation

- Root process (rank 0):
 - Uses MPI_Recv() in a loop to collect messages.
 - All other processes:
 - Use MPI_Send() to send their message to root.
 - The root then prints all the received messages.
-

Sample Output (with 4 processes)

Root process waiting for messages...

Received from process 1: Hello World from process 1

Received from process 2: Hello World from process 2

Received from process 3: Hello World from process 3

Q. 5 a. MPI program to send two numbers (array elements) per process to a Root process and print the received messages.

✓ Program: Sending Two Numbers (Array Elements) to Root

// Q5(a) - Each process sends 2 numbers to the root

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int rank, size;
```

```
    int data[2], recv_data[100]; // enough space for all
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    // Each process prepares 2 integers
```

```
    data[0] = rank * 2 + 1;
```

```
    data[1] = rank * 2 + 2;
```

```
    if (rank == 0) {
```

```
        // Root process receives from all
```

```
        printf("Root waiting for data...\n");
```

```
        // Root also includes its own data
```

```
        printf("Process 0 data: %d %d\n", data[0], data[1]);
```

```
        for (int i = 1; i < size; i++) {
```

```
            MPI_Recv(&recv_data[i * 2], 2, MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
            printf("Received from process %d: %d %d\n", i, recv_data[i * 2], recv_data[i * 2 + 1]);
```

```
        }
```

```
    } else {
```

```

// Each non-root process sends its 2 numbers
MPI_Send(data, 2, MPI_INT, 0, 0, MPI_COMM_WORLD);
}

MPI_Finalize();
return 0;
}

```

Explanation

- Each process sends 2 integers to the root.
- Root uses MPI_Recv in a loop to collect all.
- Demonstrates point-to-point communication.

Sample Output

Root waiting for data...

Process 0 data: 1 2

Received from process 1: 3 4

Received from process 2: 5 6

Received from process 3: 7 8

b. MPI program to find sum of ranks of all the processes. Implement using Point-to-Point communication as well as Collective communication routines. MPI program to parallelize serial code on Pi calculation.

You'll implement it in two ways:

- 1 Point-to-Point communication (using MPI_Send/MPI_Recv)
- 2 Collective communication (using MPI_Reduce)

(i) Using Point-to-Point Communication

// Q5(b)(i) - Sum of Ranks using Point-to-Point

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```

int main(int argc, char *argv[]) {
    int rank, size, sum = 0, recv_rank;

```

```

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (rank == 0) {
    sum = rank; // start with own rank
    for (int i = 1; i < size; i++) {
        MPI_Recv(&recv_rank, 1, MPI_INT, i, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        sum += recv_rank;
    }
    printf("Sum of ranks (Point-to-Point) = %d\n", sum);
} else {
    MPI_Send(&rank, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
}

MPI_Finalize();
return 0;
}

```

✅ (ii) Using Collective Communication (MPI_Reduce)

// Q5(b)(ii) - Sum of Ranks using MPI_Reduce

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int rank, size, sum;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```



```
MPI_Reduce(&rank, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
```

```
if (rank == 0)
```

```
    printf("Sum of ranks (Collective) = %d\n", sum);
```

```
MPI_Finalize();
```

```
return 0;
```

```
}
```

Explanation

- Point-to-point: Root explicitly receives from all others.
- Collective: MPI_Reduce automatically combines all values with MPI_SUM.

Sample Output

Sum of ranks (Point-to-Point) = 6

Sum of ranks (Collective) = 6

(for 4 processes with ranks 0+1+2+3)

c. MPI program to find sum of n integers on in which processors are arranged in ring topology using MPI point-to-point blocking communication library calls.

Each process holds an integer.

They are arranged in a ring, where each process sends its partial sum to the next process.

After one full circulation, all processes have the total sum.

Program: Sum of Integers in Ring Topology

```
// Q5(c) - Sum of n Integers in Ring Topology
```

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[]) {
```

```
    int rank, size;
```

```
    int send_val, recv_val, sum;
```

```
MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);

MPI_Comm_size(MPI_COMM_WORLD, &size);


send_val = rank + 1; // each process holds a number (1..n)
sum = send_val;


int next = (rank + 1) % size;
int prev = (rank - 1 + size) % size;


// Ring communication: pass partial sums around
for (int step = 0; step < size - 1; step++) {
    MPI_Send(&sum, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
    MPI_Recv(&recv_val, 1, MPI_INT, prev, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    sum += recv_val;
}


printf("Final sum at process %d = %d\n", rank, sum);


MPI_Finalize();
return 0;
}
```

Explanation

- Each process sends its running sum to the next process.
- It receives from its previous process.
- After (size-1) steps, each process has the global sum.

Sample Output (4 processes)

Final sum at process 0 = 10

Final sum at process 1 = 10

Final sum at process 2 = 10

Final sum at process 3 = 10

 Total = $1 + 2 + 3 + 4 = 10$, so all processes correctly hold the final sum.